App Monitor

System & Product Review

STATUS	OWNER	LAST UPDATED
Proposed	Jacob Crandall	July 11, 2026

Authors	OpenAI Codex
Reviewers	Product owner; macOS engineering; security/privacy review
Related docs	README.md; PRIVACY.md; RELEASING.md; design handoffs
Scope	Architecture, live macOS UI, safety, operations, and five redesign directions.

1. Abstract

App Monitor is a local-first macOS utility that combines app inventory, foreground usage, storage analysis, update management, reversible cleanup, uninstall planning, and action history. The core is technically viable and currently healthy: the full local gate passes 56 tests and rebuilds/signs the packaged app. The main risk is no longer missing capability; it is accumulated product and orchestration complexity concentrated in one SwiftUI screen and one main-actor model.

This review recommends an action-first shell backed by domain coordinators, a versioned operation ledger, explicit scan/update coverage, and contextual detail panes. The design remains native, single-user, and local-only on macOS 14+. It does not add accounts, a hosted backend, telemetry, or autonomous destructive behavior. External dependencies remain the local file system, macOS frameworks, Homebrew, mas, softwareupdate, and app-provided release feeds.

2. Goals and Non-Goals

Goals	Non-goals
Make the next useful action obvious within five seconds of launch.	Replace dedicated package managers, malware scanners, or whole-disk visualizers.
Keep cleanup, uninstall, and update actions previewable, reversible where possible, and fully auditable.	Add cloud sync, user accounts, cross-device history, or product telemetry.
Keep cached navigation responsive while scans and provider work run off the main actor with cancellation and coverage reporting.	Auto-delete user data or install manual/restart updates without explicit policy and review.
Ship the redesign incrementally behind a feature flag while preserving the existing SQLite database and provider test suite.	Redesign every specialist analytics view before the navigation and orchestration foundation is stable.

3. Background and Problem Statement

The live app exposes a capable but crowded three-pane workspace. The sidebar mixes top-level destinations, quick filters, update provider sources, analytics, and maintenance; the persistent right inspector continues to show the selected app even on Updates, History, and Settings. This reduces usable width and creates context mismatch. Settings is a long undifferentiated form, the global toolbar carries actions that are not relevant to every route, and large counts such as 958 warnings are not summarized by urgency. The strongest current flow is Quarantine Review because it explains the item, exact path, safety steps, and reversible queue in one place.

The implementation reflects the same breadth: DashboardView.swift is 9,405 lines, AppModel.swift is 4,396 lines with more than 50 published properties, and AppDataStore.swift is 2,663 lines with many schema responsibilities. Caching already removed redraw-time SQLite work, but AppModel remains the owner of inventory, analytics,

scanning, cleanup, updates, self-update, uninstall, scheduling, settings, selection, and history. The proposed boundary is: SwiftUI shell and route view models -> domain coordinators -> scanner/provider/executor adapters -> SQLite operation/state store and external macOS tools. The invariant is fail closed for destructive side effects and keep the UI driven by durable or explicitly transient state.

4. Proposed Architecture

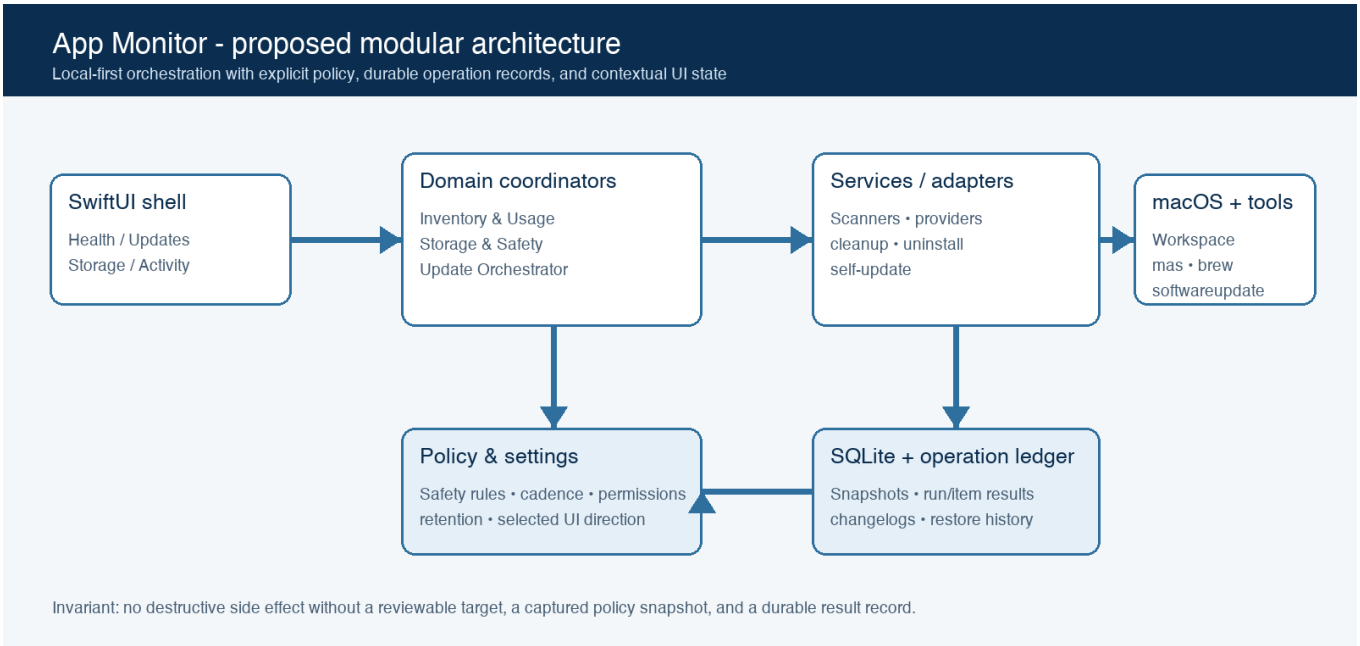


Figure 1. Proposed modular architecture for App Monitor.

CORE COMPONENTS

Component	Responsibility	Primary storage	Failure behavior
App shell + route view models	Own navigation, route-specific state, contextual selection, accessibility labels, and rendering only.	Published snapshots and transient selection	Show last valid snapshot and a route-scoped error; never block the whole shell.
Inventory & Usage coordinator	Inventory scans, tracking, timeline/summary queries, retention, exclusions, and cached usage snapshots.	apps, usage_segments, imported usage	Tracking can pause; queries fall back to last durable data with freshness shown.
Storage & Safety coordinator	Storage jobs, coverage, health findings, cleanup policy, quarantine, restore, and uninstall plans.	storage_items, findings, suggestions, action history	Permission gaps are first-class coverage states; destructive execution fails closed.

Component	Responsibility	Primary storage	Failure behavior
Update orchestrator	Normalize providers, compute eligibility, run selected updates, reconcile results, and capture changelogs.	app_updates, update runs/results, changelogs	Provider failures remain isolated and auditable; manual/restart items are never auto-selected.
Persistence + operation ledger	Versioned migrations, transactions, run/item lifecycle, settings snapshots, integrity checks, and export boundaries.	SQLite WAL database	A required write failure blocks side effects; recovery uses idempotent rerun and reconciliation.

5. Request Lifecycle

a. A manual action, scheduled timer, app activation event, or usage event enters a domain coordinator with route filters, target IDs, and current settings.
b. The coordinator normalizes paths/provider records, checks permissions and policy, rejects protected/symlinked/unsupported targets, and resolves the user-visible action type.
c. It loads the last durable snapshot plus a frozen policy/settings snapshot; the UI immediately publishes a lightweight queued/running state.
d. Scanner and provider work runs off the main actor with bounded concurrency, progress, cancellation, and provider-specific timeouts.
e. Before cleanup, uninstall, restore, or update side effects begin, a run record and selected item set are persisted; inability to write fails closed.
f. Each downstream result is recorded independently. Partial provider failure does not erase successful results, and restart/administrator requirements remain explicit terminal states.
g. The coordinator reconciles installed/file-system state, closes the run, refreshes cached view snapshots, and exposes completion, failures, and restore/retry actions in History.

6. API and Data Contracts

Primary data contract

Field	Type	Required	Description
operation_id	UUID text	Yes	Stable run identifier for scan, update, cleanup, uninstall, restore, or import.
kind	Enum	Yes	Defines policy, lifecycle, eligible targets, and valid result states.

Field	Type	Required	Description
target_id	Text	No	App, provider, path, or aggregate scope; paths are standardized before persistence.
requested_at	Timestamp	Yes	Local audit time; completed_at is stored when the run reaches a terminal state.
policy_snapshot	Versioned JSON	Yes	Frozen cadence, safety, permission, and auto-eligibility settings used by this run.
status	Enum	Yes	queued, running, completed, partial, failed, cancelled, or needsRestart.
item_results	Nested records	Yes	Per-target status, message, bytes/version transition, side-effect evidence, and recovery link.

Contract guarantees

a. A side-effecting action must have a validated target set and durable run record before execution.
b. Run IDs and item IDs are stable; reconciliation updates records instead of creating ambiguous duplicate history.
c. Provider/source, from/to versions, target path, policy version, timestamps, and terminal outcome are captured for audit and recovery.
d. SQLite is the source of truth for App Monitor observations and operations, not for external package-manager or file-system truth; reconciliation remains required.

The schema is implemented in AppDataStore migrations and should be formalized as versioned migration files with a schema_version table.

Keep migration fixtures and contract tests alongside AppMonitorCoreTests; publish human-readable field semantics in docs/.

7. Consistency, Idempotency, and Replay

App Monitor needs strong local ordering around side effects, but not distributed-system semantics. A serial datastore queue and SQLite WAL provide ordered writes; coordinators freeze policy per run and publish UI snapshots only after coherent state transitions. Read-only scans and provider checks may be repeated. Cleanup, uninstall, restore, and install actions require stable IDs, preflight validation, durable attempt records, and post-action reconciliation. Partial success is acceptable only when each item has an explicit terminal result and the UI does not imply atomic completion.

Scenario	Expected behavior	Reasoning
Repeated scan or update check	Coalesce while active or create a new run after the prior run closes; replace current snapshots deterministically.	Read-only work is safe to repeat and durable history remains unambiguous.
Run-record write fails	Do not start destructive/update execution; surface a retryable storage error.	Failing closed prevents unaudited side effects.
Provider or item partially fails	Persist per-item outcomes, mark the run partial, reconcile successful items, and offer scoped retry.	A truthful partial state is safer than rollback claims the external tool cannot guarantee.
Settings change mid-run	Continue with the captured policy snapshot; new settings apply to the next run.	Stable behavior and audit reconstruction require immutable per-run policy.

8. Security and Privacy Considerations

a. Single-user local trust boundary. Use standard macOS authorization only at the action that needs it; protected paths and Apple/system apps remain blocked.
b. Usage history, paths, app inventory, and action history are sensitive. Keep them local, redact secrets from messages, and never add telemetry without a separate explicit design review.
c. Credentials belong in Keychain or ephemeral authorization context only, never SQLite, UserDefaults, process arguments, exports, or logs. Self-update packages require checksum/signature validation.
d. Cleanup defaults to quarantine; uninstall defaults high-risk paths off; symlink destinations are not followed; direct-download actions allow only trusted URL schemes and explicit user review.
e. Add pause tracking, per-app exclusion, clear-history, and 30/90/365-day/forever retention. Show scan coverage and permission gaps so totals are not presented as complete when access is partial.

9. Operational Readiness

Signal	SLO or alert	Owner	Launch gate
Navigation response	Target p95 <100 ms from click to cached content; no SQLite/file scan in SwiftUI body.	App Monitor	Required
Progress visibility	Visible progress/freshness within 250 ms; cancellation acknowledged within 1 s.	App Monitor	Required

Signal	SLO or alert	Owner	Launch gate
Provider completion	>=95% successful provider checks excluding unavailable/uninstalled tools; failures isolated by source.	Update orchestrator	Required
Audit completeness	100% of side-effecting item attempts have run ID, target, policy snapshot, and terminal result.	Operation ledger	Required
Restore/integrity	100% deterministic fixture restores; zero silent migration or relationship loss in upgrade tests.	Storage & persistence	Required

Rollout: feature-flag the new shell; seed deterministic data; run current/new shells against the same database; preserve a one-release rollback path; promote only after accessibility and destructive-action audit gates pass.

10. Alternatives Considered

Alternative	Why it was considered	Why it was not selected
Keep extending the current shell	Lowest short-term implementation cost.	Perpetuates context mismatch and large AppModel/DashboardView blast radius.
Adopt a full architecture framework	Strong reducer/state conventions and testability.	Large migration cost; domain coordinators deliver most value without framework lock-in.
Split into separate usage, cleanup, and updater apps	Each product becomes easier to explain.	Loses the core differentiator: usage, storage, risk, and update context together.
Add a cloud backend	Cross-device sync and remote analytics become possible.	Conflicts with local-first privacy, adds account/security operations, and is unnecessary for the current goal.

11. Open Questions

a. Which of the five UI directions should define the new shell and primary navigation model?

b. Should the inspector disappear on non-app routes, collapse by default, or become a route-specific drawer?

c. What is the default usage-history retention period, and should excluded apps be dropped at capture time or filtered later?
d. Which update classes may ever run automatically: managed Homebrew only, App Store, direct-download, or none without confirmation?

12. Decision and Next Steps

Adopt an action-first product shell, select one of the five visual directions as the source target, and refactor behind it rather than rewriting the core. The recommended architecture extracts Inventory & Usage, Storage & Safety, Updates, and Persistence/Operations coordinators from AppModel; decomposes DashboardView into route files; adds a versioned operation ledger, cancellation, permission/coverage states, and deterministic preview data; then rolls the chosen shell out behind a feature flag while the existing packaged app remains the rollback path.

Milestone	Deliverable	Exit criteria
M1	Extract route files and domain coordinators; preserve behavior and current database.	56 tests and scripts/ci pass; no route performs datastore/file work in body; cached navigation target met.
M2	Versioned migrations, operation ledger, scan coverage, cancellation, retention/exclusion settings.	Failure/replay fixtures pass; destructive attempts are 100% auditable; permission gaps are visible.
M3	Implement the selected UI direction behind a feature flag with deterministic demo state.	Screenshot QA, keyboard traversal, VoiceOver labels, contrast, narrow-window, light/dark tests pass.
M4	Refine specialist Update, Storage, Activity, and History workspaces and retire the legacy shell.	Stable beta period; no P0/P1 safety regressions; rollback tested; product metrics accepted.

Appendix A. Product Design Audit

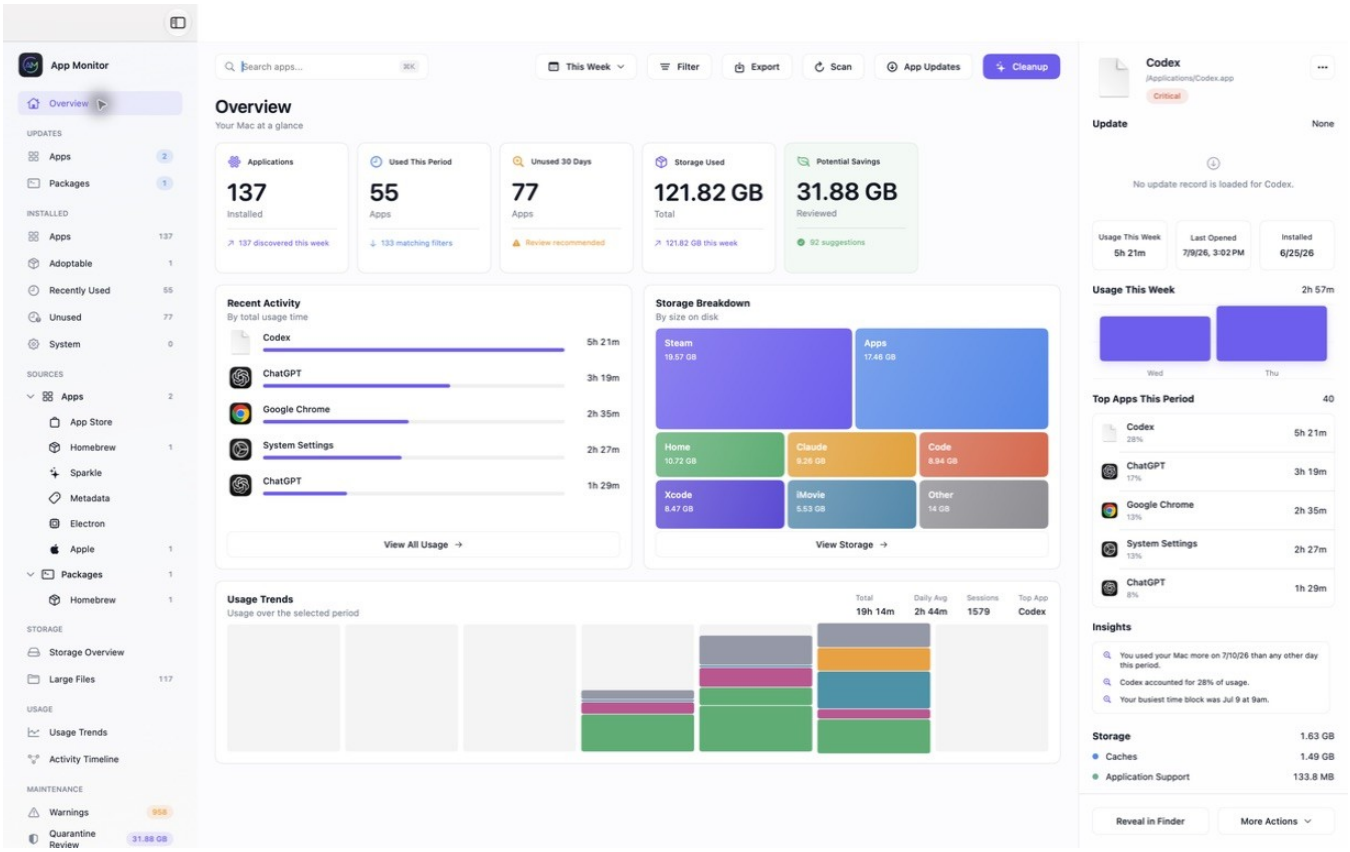
Overall verdict: the product is capable and unusually transparent for a Mac maintenance utility, but the shell presents too many equal-weight destinations and keeps app-specific detail visible on routes where it is irrelevant. The best existing design pattern is Quarantine Review because it connects evidence, safety, and action.

Highest-impact changes

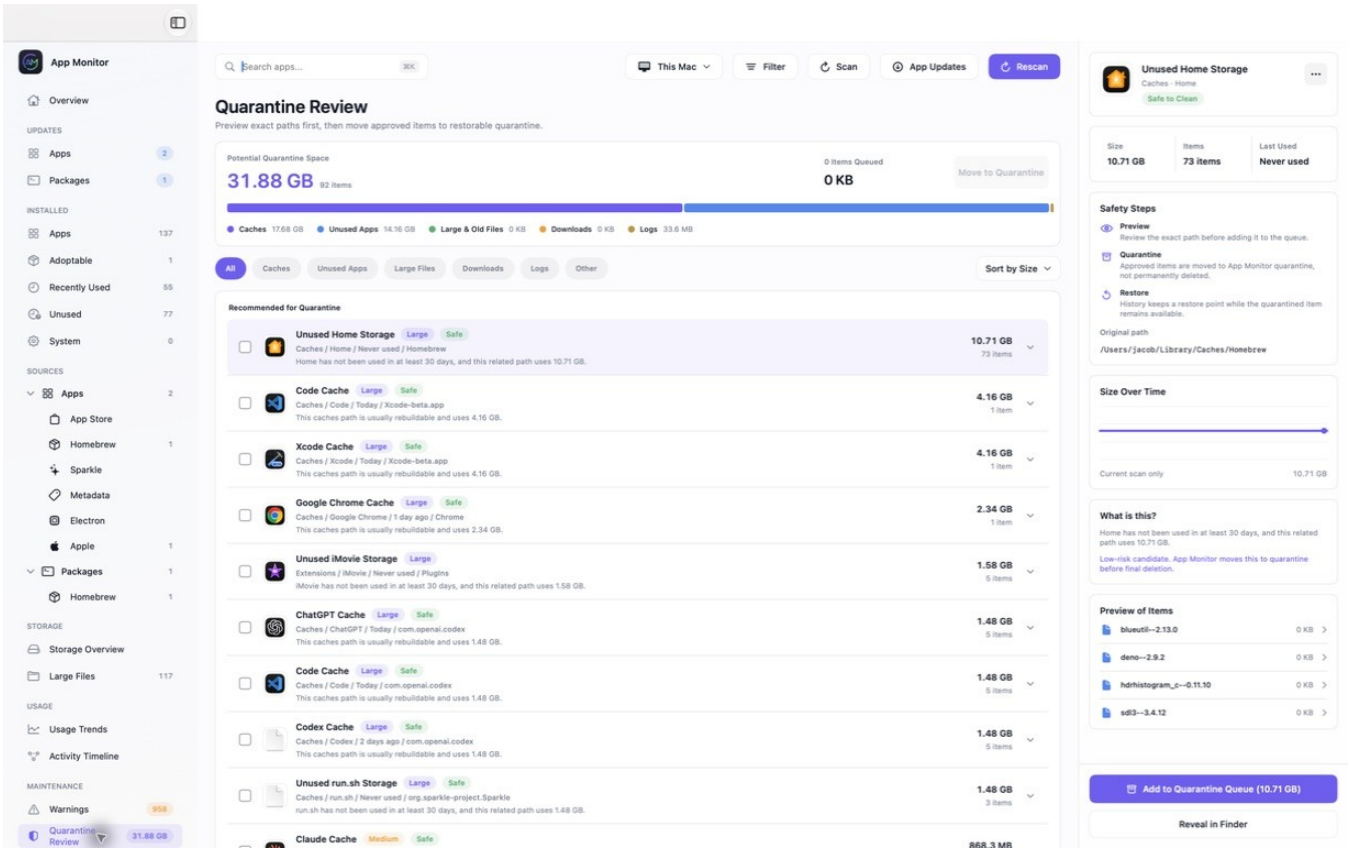
- P0 — Replace the destination inventory with four product areas: Health, Updates, Storage, and Activity. Keep provider/source filters inside the relevant workspace.
- P0 — Make the inspector contextual. Hide it on Settings and Overview; use a route-specific drawer on Updates, History, and Quarantine Review.
- P0 — Replace multiple global toolbar actions with one contextual primary action plus overflow. Search/date/filter remain global only where they affect the current route.
- P0 — Split Settings into Appearance, Tracking & Privacy, Scanning, Updates, and Advanced. Add pause tracking, app exclusions, retention, clear-history, and permission coverage.
- P1 — Turn 958 warnings into an actionable summary by severity, affected app, confidence, and recommended next action. Do not use a raw count as the primary signal.
- P1 — Keep long-running work in a compact status center. Progress must not push every route downward or disable unrelated inspection.
- P1 — Use progressive disclosure for changelogs and history. Keep the current task visible and open detailed timelines only on selection.
- P1 — Audit small labels, pale gray text, icon-only controls, focus order, keyboard hit targets, and non-color state labels in both light and dark themes.
- P1 — Say “space available for review,” not guaranteed savings. Show freshness and permission coverage next to totals.
- P2 — Scope search and filters explicitly so users know whether they affect apps, providers, storage paths, or usage analytics.

Evidence and limits

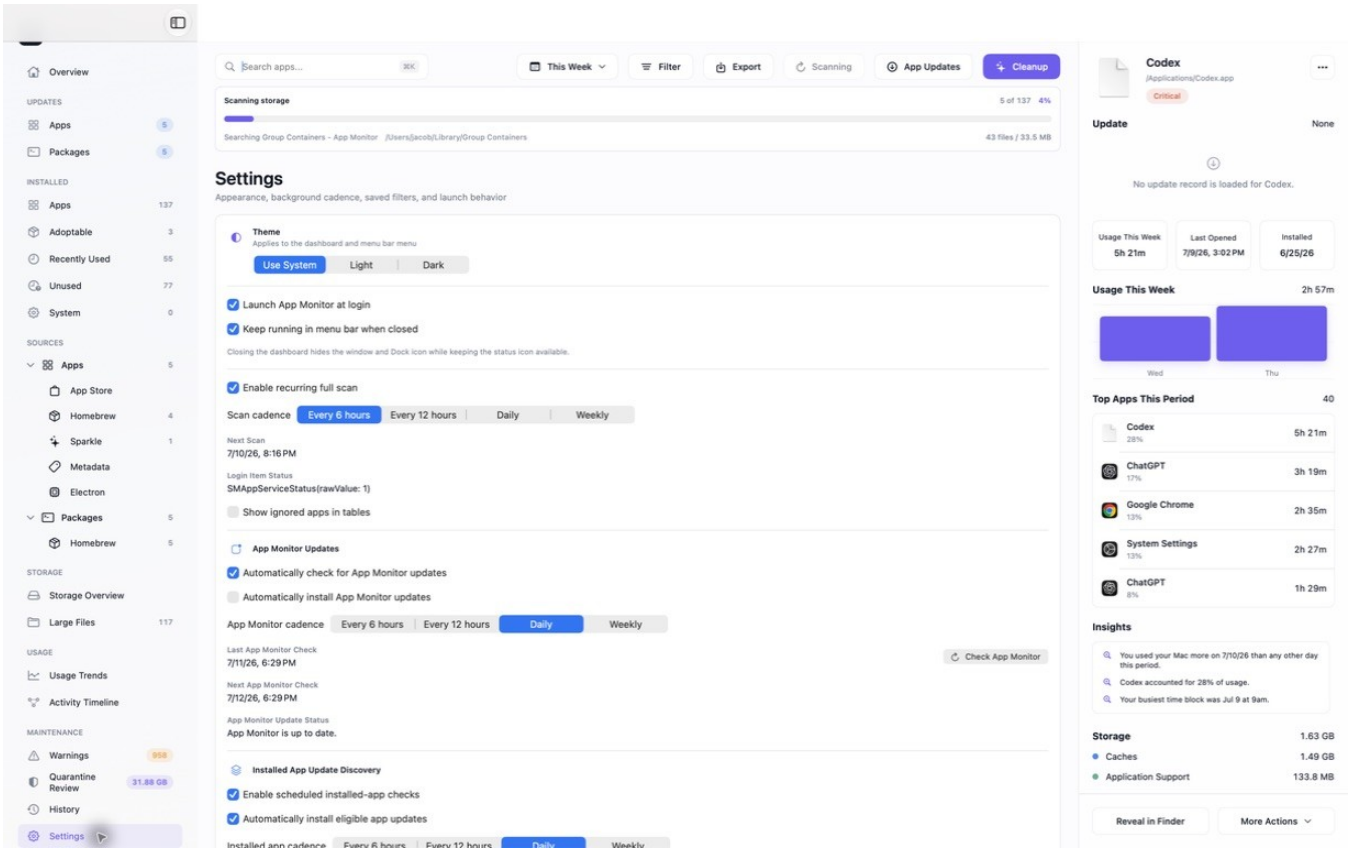
Evidence comes from a live July 11, 2026 capture of Overview, Installed App Updates, Quarantine Review, History, Settings, and Usage Trends in the packaged /Applications build. Screenshots support layout, hierarchy, wording, density, and visible state findings. They do not prove VoiceOver traversal, measured contrast, destructive-action correctness, permissions behavior, or narrow-window behavior; those require instrumented testing.



Current Overview — useful breadth, but five metric cards, two content areas, usage analytics, a dense sidebar, global toolbar, and a persistent inspector compete at once.



Current Quarantine Review — strongest flow: exact target, size, safety steps, preview, reversible queue, and a clear selected-item action.



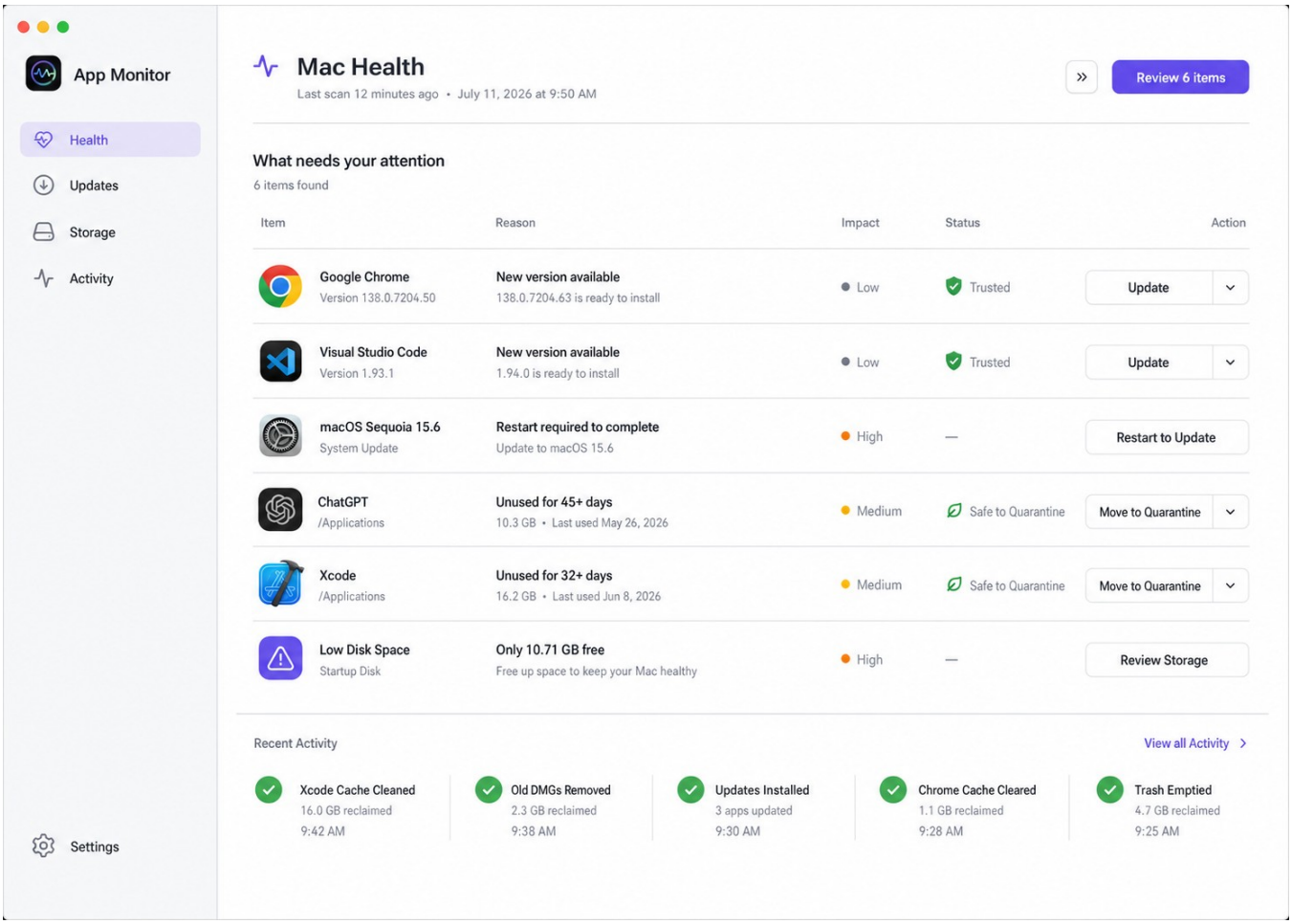
Current Settings — the persistent app inspector is unrelated, while update and scan controls form one long page without category navigation.

Appendix B. UI Revision Directions

All five directions are grounded in the live product and preserve local-first, safety-first behavior. The choice should be based on the product's primary promise, not surface styling alone.

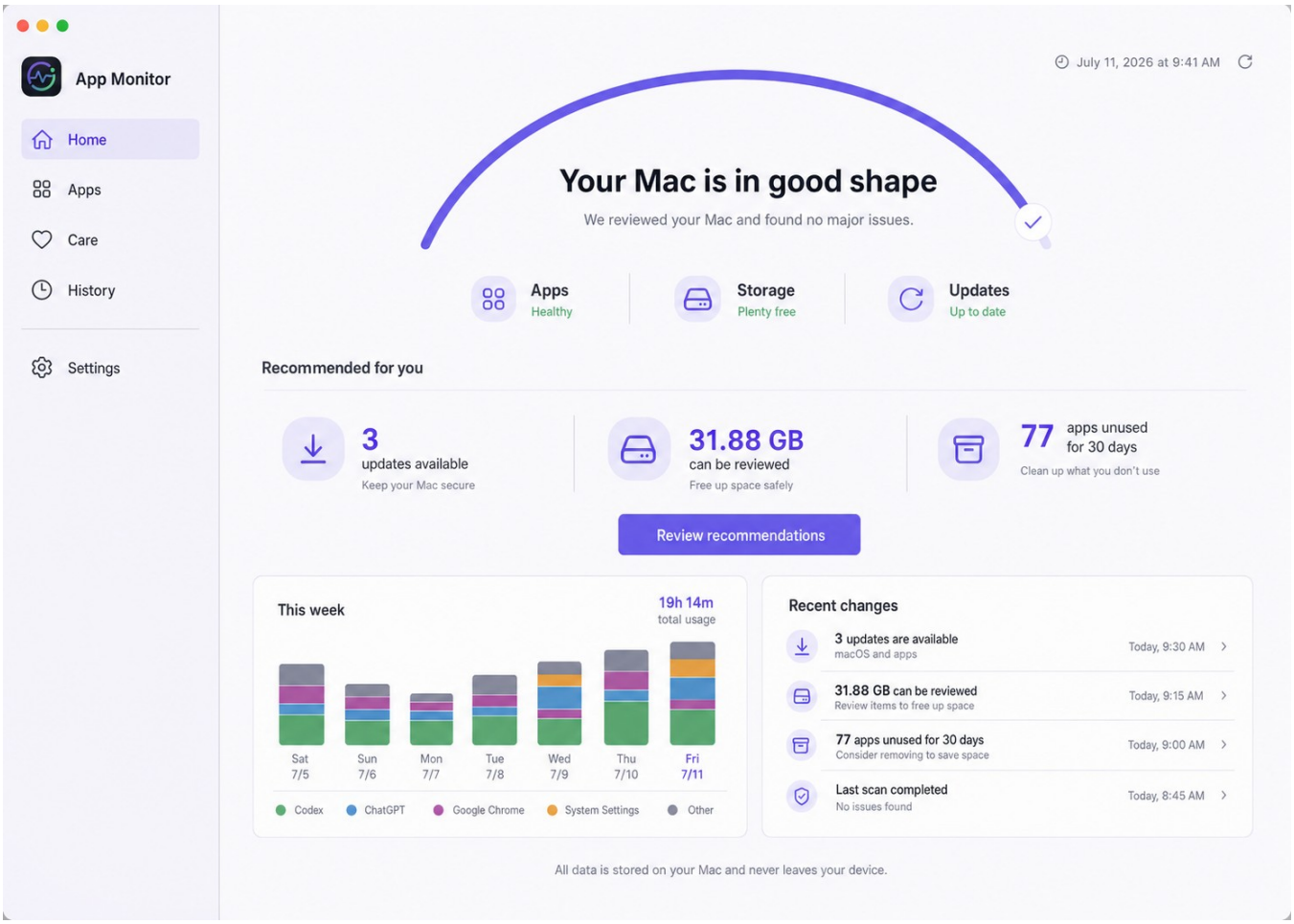
Direction 1 — Operations Console

Best for a product promise centered on "what needs attention now." Strongest synthesis of updates, warnings, and safe maintenance; requires careful prioritization rules.



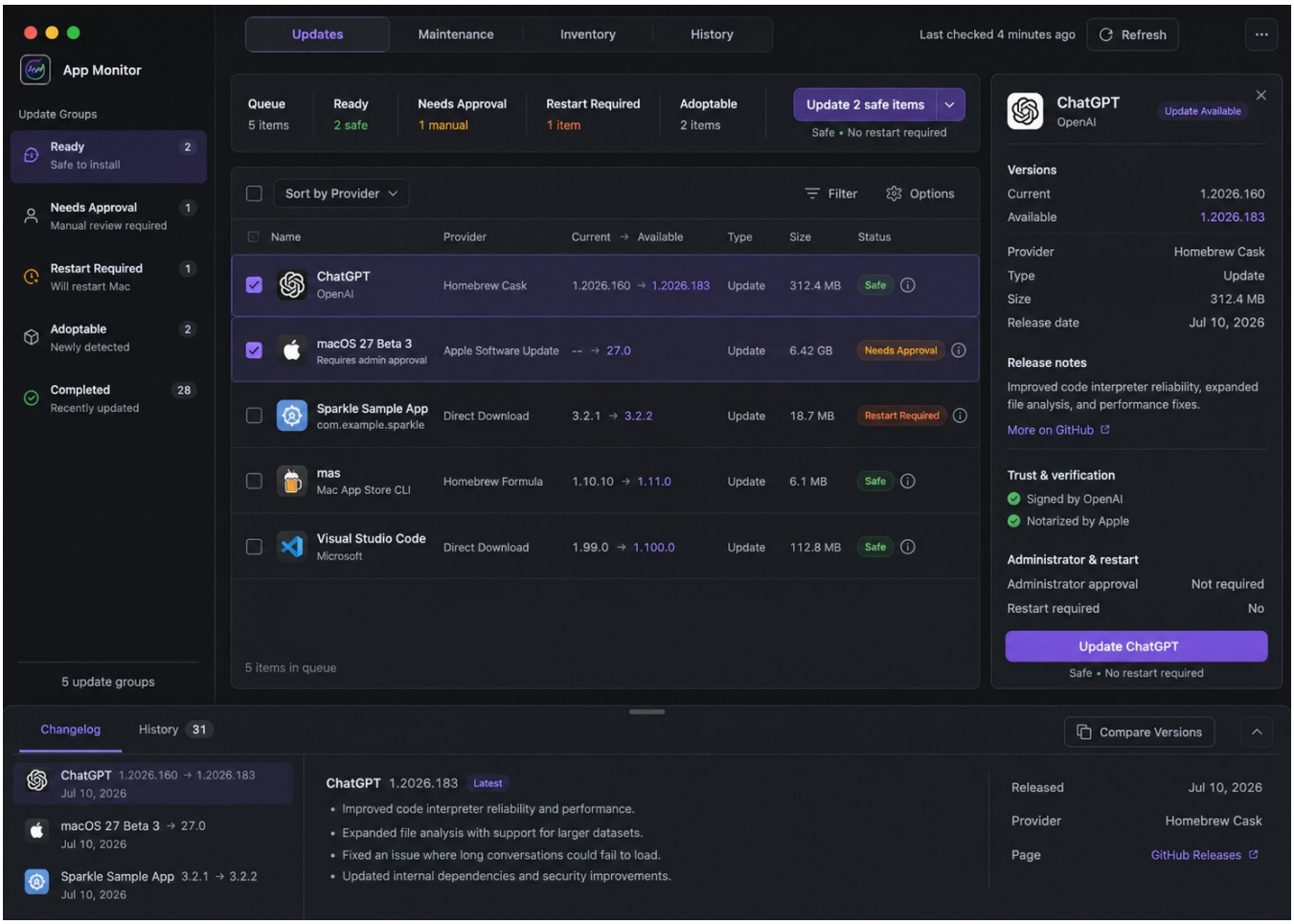
Direction 2 — Calm Mac Health

Best for mainstream users who want reassurance and guided recommendations. Lowest cognitive load; risks hiding expert detail too early.



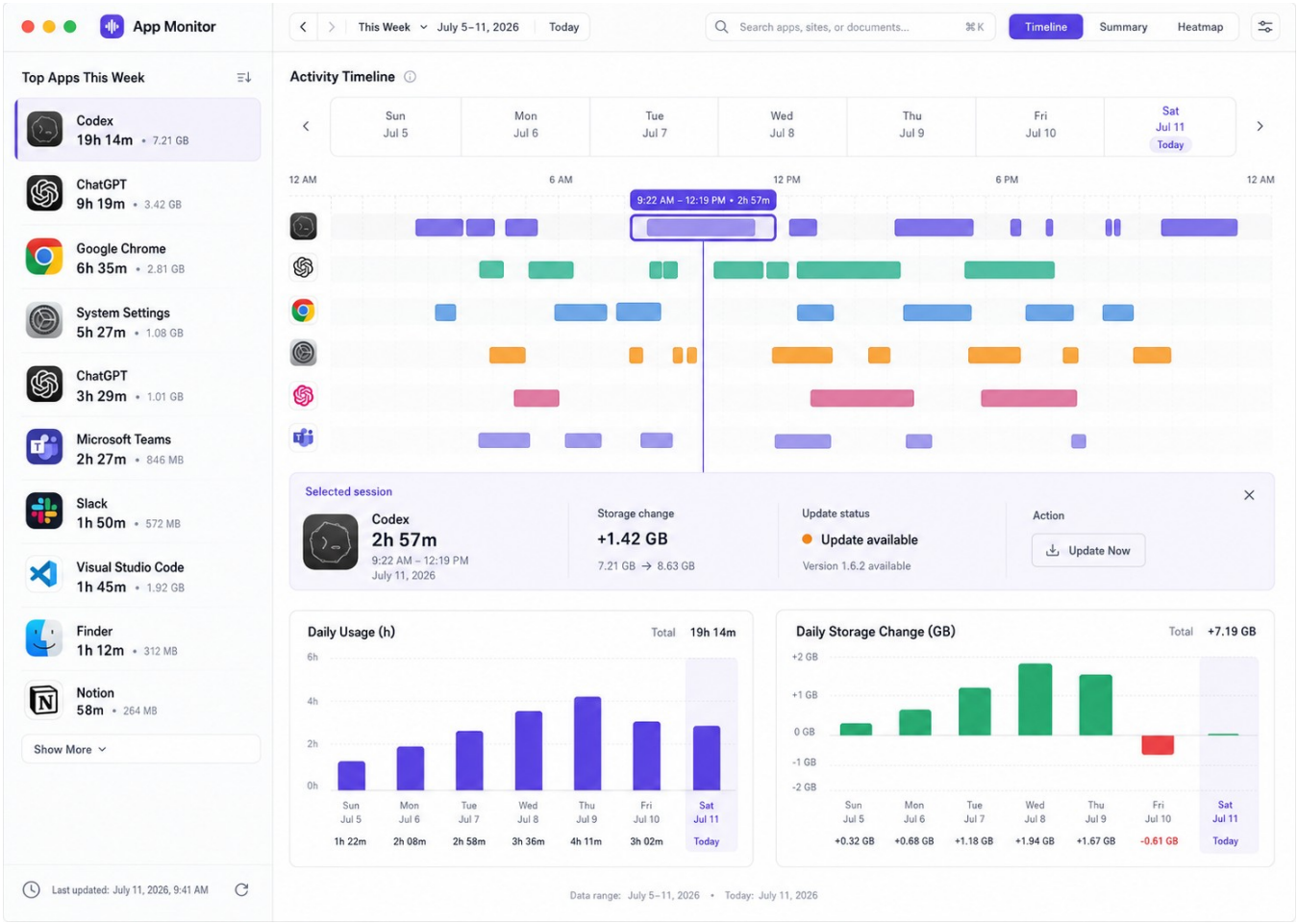
Direction 3 — Update Command Center

Best if broad update management becomes the lead differentiator. Excellent provider/release-note clarity; de-emphasizes usage and storage context.



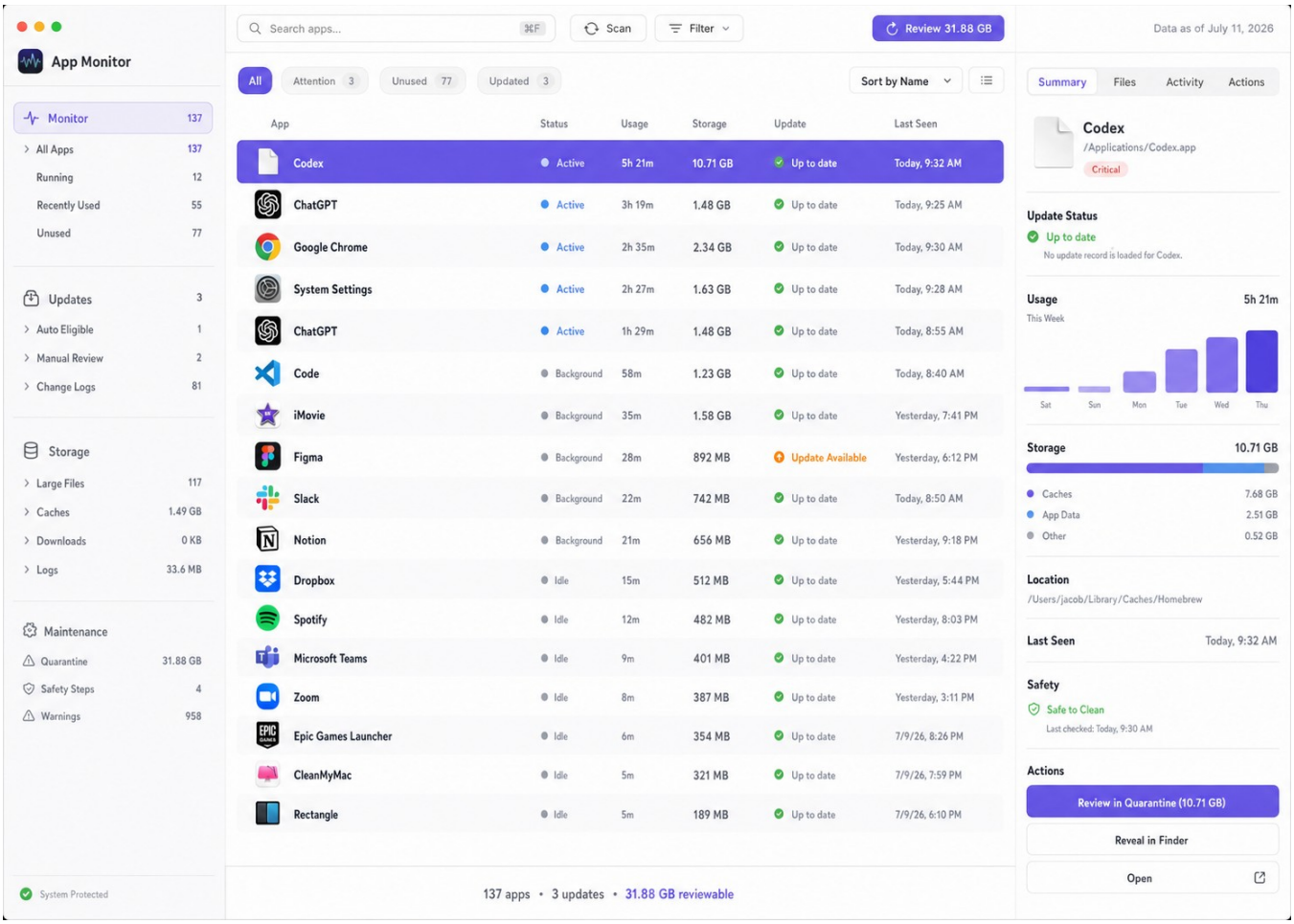
Direction 4 — Activity Timeline Workspace

Best if personal usage intelligence becomes the lead product. Strong coordinated analysis; requires first-class retention/privacy controls.



Direction 5 — Compact Pro Utility

Best for expert users who value density, keyboard speed, and Finder-like inspection. Scales well; has the highest accessibility and onboarding burden.



Appendix C. Prioritized Implementation Backlog

The backlog separates product clarity from deep infrastructure work so the team can improve the experience without compromising safety or creating a rewrite-sized release.

Priority / effort	Change	Success measure
P0 / M	Contextual inspector + four-area navigation	Route relevance; +15–20% usable width; task recognized in <5 seconds.
P0 / M	Split DashboardView and AppModel into route/domain files	Files become reviewable; route changes no longer touch unrelated workflows.
P0 / M	Versioned operation ledger and fail-closed side effects	100% destructive/update attempts reconstructable by run and item.
P0 / S	Settings categories + privacy controls	Pause, exclusions, retention, clear history, and permission coverage are discoverable.
P1 / M	Cancelable scan jobs + compact status center	Progress visible <250 ms; cancellation <1 s; unrelated routes remain usable.
P1 / S	Warning severity/confidence/action model	Raw 958 count replaced by prioritized, explainable review groups.
P1 / M	Deterministic demo database + screenshot states	Reliable empty/loading/error/dense/light/dark visual regression coverage.
P1 / M	Accessibility validation matrix	Keyboard, focus, VoiceOver labels, contrast, non-color states, and large text pass.
P2 / L	Incremental scans and persisted storage rollups	Fast category/app refresh; historical deltas without loading all paths.